



Assessment-3
Research Paper
(Predicting Online Gaming Behaviour)

Submitted to

Dr. Lu Qin

Course Name – Studio-3 (Innovation)

Subject Code - 42048

Session – Spring 2024

Prepared by

Group 3

Alvaro Perico 13729090

Fabiha Rahman 24689349

Haoran Fang 12662173

Qimin Yin 13229277

Paras Sharma 24697397

Spenser Gautama 13091323

Table of Contents

Introduction.....	2
Introduction to Business Context	2
Problem Statement.....	2
Project Origin & Motivations	2
Project Contribution	3
How is this paper organized	3
Scoping the Project	3
Project Aim	3
Project Objectives.....	4
Project Deliverables.....	4
Stakeholder Analysis	5
Defining Success Criteria	7
Methodology.....	8
Methodology Framework	8
Data Understanding.....	8
Data Description	8
Exploratory Data Analysis	10
Data Preparation.....	14
Data Cleaning.....	14
Feature Engineering.....	18
Data Transformation.....	18
Feature Selection	20
Model Selection	22
Experiments.....	25
Experiment Plan	25
Splitting Dataset.....	26
Applying SMOTE to address class imbalance	26
Model Development & Training.....	27
Model Validation & Evaluation.....	34
Conclusion & Discussion	36
Possible Limitations	36
Future Recommendations for Experiments.....	37
References & Appendices.....	37
Contribution Table.....	38

Introduction

Introduction to Business Context

Game behavior encompasses the actions and habits players display while engaging with video games. This includes a range of activities influenced by individual personalities, game design, difficulty levels, and the investment of time and money. Our project focuses on leveraging game behavior to enhance customer engagement and sales forecasting in the retail sector.

By analyzing players purchase histories, preferences in game categories, and gameplay duration, we can offer targeted promotions and personalized recommendations. Predictive analytics allowed us to forecast product demand, optimizing user behavior to improve customer experience and increase market share.

Moreover, understanding player's game behavior enables us to create distinct segments, facilitating tailored marketing strategies. Identifying customers at risk of churning through behavioral analysis allows for proactive retention efforts. Overall, our project aims to enhance revenue by pinpointing key drivers of player's satisfaction and loyalty.

Problem Statement

The gaming industry struggles with player retention, with current rates at approximately 40% and churn at 25%. This leads to lost revenue and diminished player engagement. Traditional marketing strategies often overlook the nuanced preferences of gamers, resulting in ineffective promotions. Our project seeks to analyze player behavior using data analytics and machine learning to identify key drivers of loyalty and engagement, enabling tailored strategies that reduce churn and enhance the overall gaming experience.

Project Origin & Motivations

The Gaming Behavior project originated from the rapid evolution of the online gaming industry, which has witnessed a massive influx of players across various platforms. As the popularity of gaming grows, understanding player behavior becomes essential for enhancing user experience and optimizing business outcomes. Key motivators for this project include:

Data Explosion: The extensive volume of player-generated data—including gameplay statistics, session duration, and purchase history—provides opportunities for in-depth analysis. This information can inform the development of predictive models that anticipate player preferences and behaviours.

Improving Player Retention: With the gaming industry facing challenges from high churn rates, this project focuses on identifying the key factors influencing player loyalty and implementing strategies to enhance retention rates.

Enhancing Player Engagement: By analyzing player behavior, the project aims to personalize gaming experiences through targeted recommendations and content adjustments, fostering a stronger connection between players and games.

Optimizing Marketing Strategies: Understanding player preferences allows for the development of tailored marketing approaches, improving user acquisition and maximizing revenue through effective promotional strategies.

Project Contribution

This project enhanced the marketing division's strategy by identifying key drivers of player engagement and predicting loyalty. The goal is to boost revenue by improving player satisfaction.

We used linear and tree-based machine learning algorithms, specifically logistic regression, K-nearest neighbour and random forest, to uncover influential factors affecting satisfaction. These models were trained on historical data and evaluated on unseen datasets to ensure effectiveness. Results are detailed in the Experiments section.

A key contribution occurred during the experimentation phase, where we segmented players based on game preferences to assess shifts in feature importance and improve accuracy. This provided us insights into whether marketing strategies need customization.

Additionally, the project aims to redefine the perception of our gaming platform from a typical service to an industry innovator. By implementing factors that drive player satisfaction, we can enhance our reputation and increase player loyalty.

How is this paper organized

This research paper is structured systematically to highlight different aspects of gaming behavior. It follows the CRISP-DM methodology to tailor the content to the reader's needs. The organization of the paper is aligned with the format presented in the table below.

Research Paper Section	CRISP-DM Stage
Introduction	Business Understanding
Scoping the Project	Business Understanding
Methodology	Data Understanding
	Data Preparation
Experimentation	Data Modelling
	Evaluation
Discussion & Conclusions	Deployment

Table-1: Organization of Research Paper

Scoping the Project

Project Aim

This project analyses historical data from an online gaming platform to predict player behaviours such as churn rates, in-game purchases, and level completion patterns. By

examining factors like playtime, session frequency, and engagement levels, we aimed to develop models that enhance retention strategies and optimize gameplay. We also identified engagement patterns, leveraging insights to improve user retention and inform monetization strategies. Ultimately, the goal was to create a personalized gaming experience tailored to player preferences.

Project Objectives

Data Collection and Preparation

The first step involves gathering comprehensive player data, including demographics, session logs, purchase history, and player preferences from the gaming platform. This data underwent cleaning and standardization to ensure accuracy and consistency, preparing it for effective modeling.

Feature Engineering and Selection

After preparing the data, relevant features such as gameplay duration, session frequency, and purchase categories were identified. We enhanced the dataset by selecting key variables and removing redundant ones, optimizing it for predictive analysis.

Model Development

A range of machine learning models were developed to forecast critical player behaviours, such as churn rates and in-game purchases. The models were trained on the cleaned dataset to recognize patterns and make accurate predictions based on player activity.

Validation and Testing

To ensure robustness, the developed models were validated using a separate testing dataset. This step confirmed that the models can generalize well across different player segments and various gaming scenarios.

Deployment

A user-friendly recommendation system was created to suggest personalized game content and promotions based on individual player habits. This system was integrated into the gaming platform, enhancing user engagement and satisfaction.

Documentation and Reporting

Finally, comprehensive documentation was prepared, detailing the entire project process. This included data sources, model development, training methodologies, and performance evaluations. The report served as a resource for understanding the project and its outcomes, facilitating potential future applications.

Project Deliverables

The deliverables for this project integrated the research and technical objectives outlined above into two key outputs aimed at addressing the business problem:

Deliverable	Description
Predictive Model	A proof-of-concept machine learning model developed in Python, designed to predict player behaviour, including churn rates and in-game purchase likelihood. This model will leverage historical gaming data to uncover insights into player engagement patterns and enhance retention strategies.
Comprehensive Report	A detailed technical report documenting the entire project process, including data collection methods, feature engineering, model development, and evaluation metrics. This report will also analyse the implications of the findings on business operations, provide recommendations for improving player engagement, and present actionable insights for stakeholders and decision-makers.

Table-2: Project Deliverables

Stakeholder Analysis

Some of the most significant stakeholders in this research are:

- **Online Gamers:** Online gamers are the main focus of this research. Their behavior significantly impacts the study's outcomes.
- **Game Developers:** Game developers influence research through their game designs, which shape player experiences. Their insights into player-game interactions are essential.
- **Academic & Industry Researchers:** These stakeholders enhance research quality by providing methods and frameworks. Their expertise helps interpret behavioral data and draw conclusions.
- **Tech Companies & Platforms:** These organizations improve gaming platforms and provide insights into player behavior using data on usage patterns and technology effects.
- **Data Analytics Firms:** These firms analyze behavioral data to find patterns that can boost player engagement and retention, helping improve the gaming experience.
- **Publishers:** Publishers influence marketing strategies and game commercialization, affecting how games are perceived and engaged with by players.

Stakeholder	Project impact	Project influence	Important to stakeholder	Project Contribution	Stakeholder engagement strategy
Online Gamers	High	High	-Enhanced gameplay experience -High quality of services	-Providing feedback through multiple surveys, forums, etc	-Consistent engagement in feedback channels
Game Developers	High	High	-Game performance improvement -Player engagement	-Implementing feedback from users to provide in-game design & mechanic improvement.	-Collaborative discussion for games updates
Academic & industry research	Medium	Medium	-High quality research insights and publications	-Offering research methodology and theoretical framework - Publishing insights for industry use	-Engagement in research partnerships and co-authored papers
Tech Companies & Platforms	Medium	Medium	-Enhanced platform performance -Player retention	-Providing platform-level data for analysis	-Technical analysis -Sharing platform data for analysis
Data Analytics Firm	High	Medium	-Ability to provide insights for better player engagement and retention	-Providing deep insight through data analysis and tools for gaming behaviour	-Regular report and data sharing such as joint analysis and meeting
Publisher	Medium	High	-Revenue and marketing strategies	-Providing market insight and financial support	- Consistent quarterly report and review meeting on user behaviour

Table-3: Stakeholder Analysis Chart (Predicting Online Gaming Behaviour Dataset)

Defining Success Criteria

Numerous elements play a crucial role in determining a project's success. For this particular initiative, several key factors are instrumental in achieving its goals and objectives:

Accuracy and Reliability: The model's effectiveness in predicting player engagement and behavior is crucial. Achieving high accuracy reduced misclassification rates, ensuring that the insights gained are both reliable and actionable for developers and marketers.

Operational Efficiency: The implementation of the predictive model streamlined game development and marketing strategies, leading to a reduction in the time required for manual analyses. Success was evaluated based on a decrease in decision-making time and an enhanced ability to swiftly respond to player behavior trends.

Cost Savings and Resource Optimization: The model facilitated significant cost savings by pinpointing high-risk areas for player churn and optimizing marketing expenditures. By effectively targeting engagement strategies, resources were allocated to maximize impact, ensuring that marketing efforts yield the best possible return on investment.

Improved Player Outcomes: The project's success was measured by enhanced player engagement and retention. We aimed for a 20-30% increase in playtime, extending average active days from 10 to 12 days per month. Additionally, we seemed a 15% increase in in-game purchases by the end of the next quarter, which will raise the average revenue per user (ARPU). By tracking critical performance indicators (KPIs) like player engagement and satisfaction, we aim to reduce the risk of underperforming major game updates by 25%, ensuring alignment with player preferences.

Methodology

Methodology Framework

Our project utilized the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology, a comprehensive framework for data-driven projects. It consists of six stages: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, and Deployment. To enhance collaboration, rapid feedback, and iterative refinement, we have also integrated Agile methodologies into our approach. A concise overview of these steps is illustrated in below figure. This structured approach ensured a robust and flexible framework for predicting player behavior based on our dataset from Kaggle.

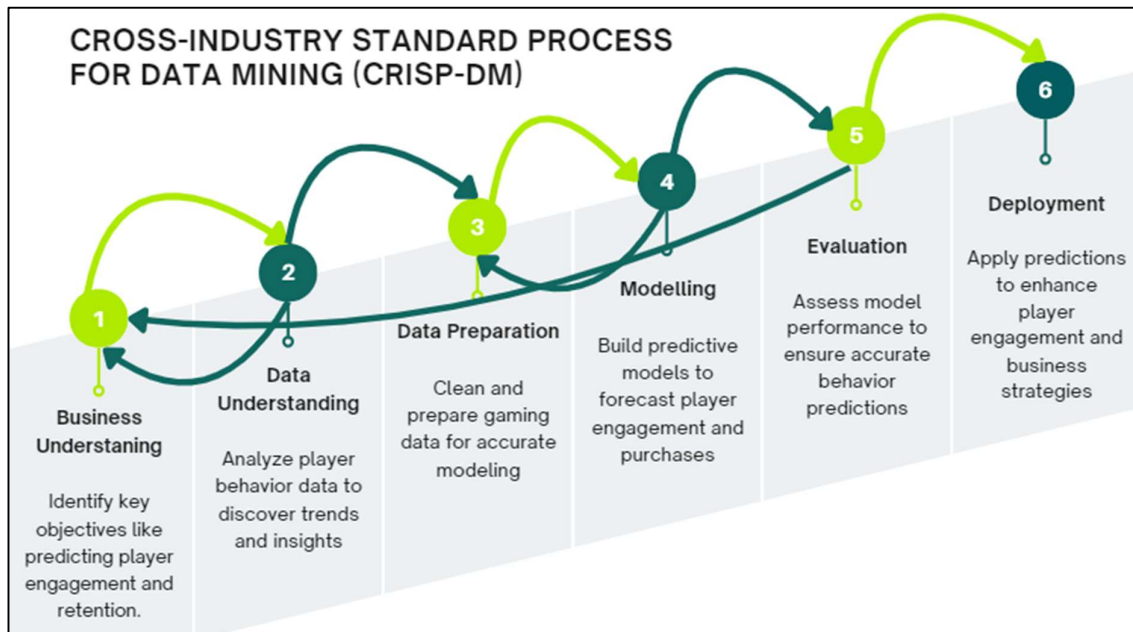


Figure-1: CRISP-DM (Online Gaming Behaviour Dataset)

The dataset, sourced from Kaggle, forms the core of this project, which aims to predict player behavior based on historical data to enhance in-game recommendations and retention strategies. The data understanding phase involved exploring the dataset to uncover patterns and trends. In the data preparation phase, we focused on data cleaning, feature engineering, and scaling to ensure the data is ready for accurate modeling and analysis.

Data Understanding

Data Description

The dataset obtained from Kaggle served as the primary source for analyzing player engagement in an online gaming platform. It consisted of approximately 40,033 entries, each representing behavioral information that can be leveraged to predict engagement levels—categorized as low, medium, and high. This analysis aimed to facilitate more effective in-game recommendations and retention strategies for developers.

The dataset captures a range of metrics related to demographics, gameplay habits, and purchase behavior, with 13 features in total. Each entry reflects a player's attributes over a specified period, with certain features subject to varying amounts of missing data, which were addressed during the data cleaning process.

The dataset encompasses various types of variables:

- **Numerical Variables:** Examples include playtime hours and average session duration.
- **Categorical Variables:** Examples include player gender, game genre, and engagement level.
- **Text Variables:** Examples include player ID and location information.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 40034 entries, 0 to 40033
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   PlayerID                             40034 non-null  int64
1   Age                                   40034 non-null  int64
2   Gender                               40034 non-null  object
3   Location                             40034 non-null  object
4   GameGenre                            40034 non-null  object
5   PlayTimeHours                        40034 non-null  float64
6   InGamePurchases                      40034 non-null  int64
7   GameDifficulty                       40034 non-null  object
8   SessionsPerWeek                      40034 non-null  int64
9   AvgSessionDurationMinutes            40034 non-null  int64
10  PlayerLevel                           40034 non-null  int64
11  AchievementsUnlocked                 40034 non-null  int64
12  EngagementLevel                       40034 non-null  object
dtypes: float64(1), int64(7), object(5)
memory usage: 4.0+ MB
```

Figure-2: Dataset Description

Also, here is the brief description about each of the feature present in our dataset in the below figure:

Features	Decription
PlayerID	Unique identifier for each player.
Age	Age of the player.
Gender	Gender of the player.
Location	Geographic location of the player.
GameGenre	Genre of the game the player is engaged in.
PlayTimeHours	Average hours spent playing per session.
InGamePurchases	Indicates whether the player makes in-game purchases (0 = No, 1 = Yes).
GameDifficulty	Difficulty level of the game.
SessionsPerWeek	Number of gaming sessions per week.
AvgSessionDurationMinutes	Average duration of each gaming session in minutes.
PlayerLevel	Current level of the player in the game.
AchievementsUnlocked	Number of achievements unlocked by the player.
EngagementLevel	Categorized engagement level reflecting player retention ('High', 'Medium', 'Low').

Table-4: Brief Description about each feature in our dataset

Exploratory Data Analysis

The exploratory data analysis (EDA) stage aimed to uncover initial patterns, relationships, and potential data issues within the online gaming dataset of 40,033 entries. This process provided insights into player demographics, gameplay behaviours, and purchase habits, helping to inform the subsequent modeling steps.

Target Variable Distribution (Counter Plot & Pie Chart)

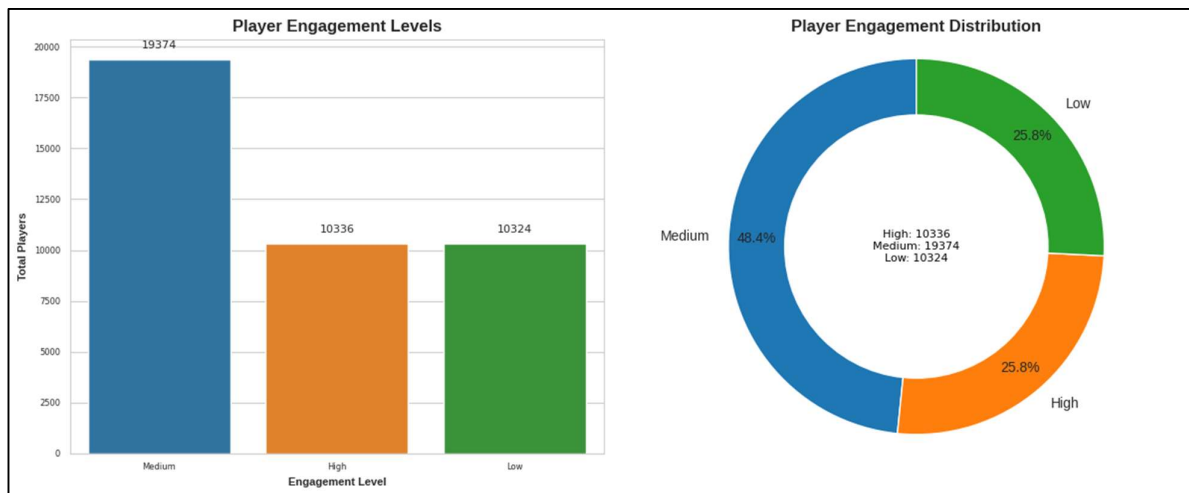


Figure-3: Target Variable Distribution (Counter Plot & Pie Chart)

Visual analysis revealed a slight class imbalance in the target variable, with more players in the "Medium" engagement level compared to "High" and "Low." To improve model performance, we used SMOTE to balance these classes during Experimentation phase.

Distribution of Gender of Players

From below chart, we observed that most players in the dataset are male, a significant factor in predicting gaming behavior. Such insight aids in tailoring models to better capture male player preferences.

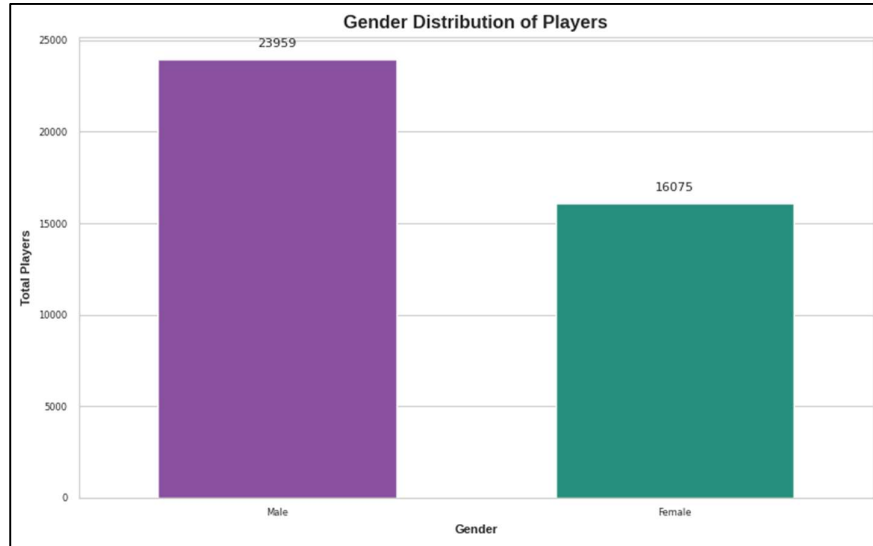


Figure-4: Distribution of Gender of Players (Counter Plot)

Distribution- Age of Players

Players are grouped into age categories: 0-17, 18-24, 25-34, 35-44, and 45-54. This age grouping, engineered from the original age variable, showing that most players are in the 25-44 range, indicating high engagement among middle-aged adults and a target audience focus within this group.

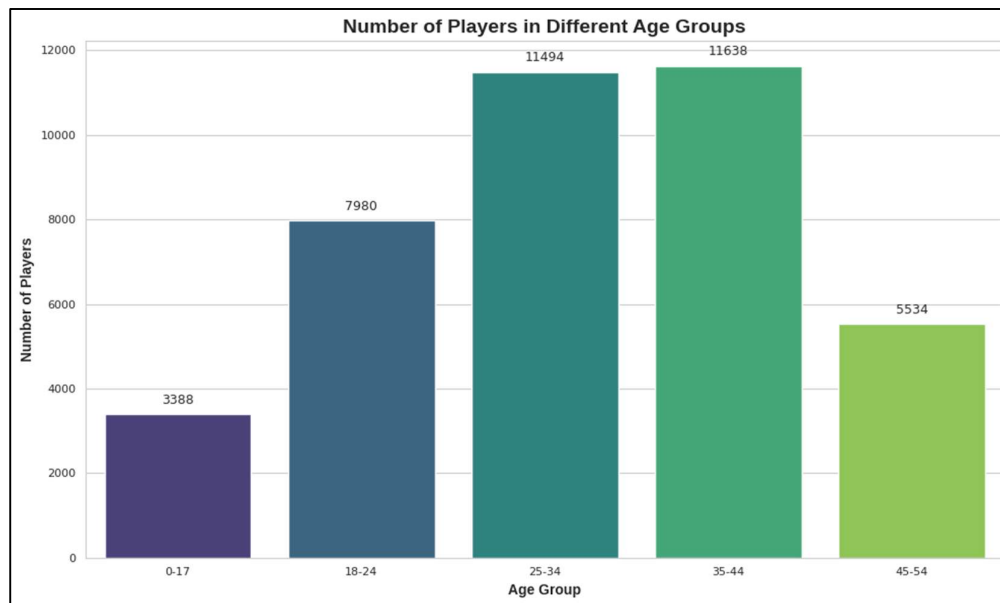


Figure-5: Distribution- Age of Players

Distribution- Gaming Session

The chart given below displays the distribution of gaming sessions per week, with a peak around 4-5 sessions. The KDE curve suggests a normal distribution, indicating that most players had a consistent gaming pattern.

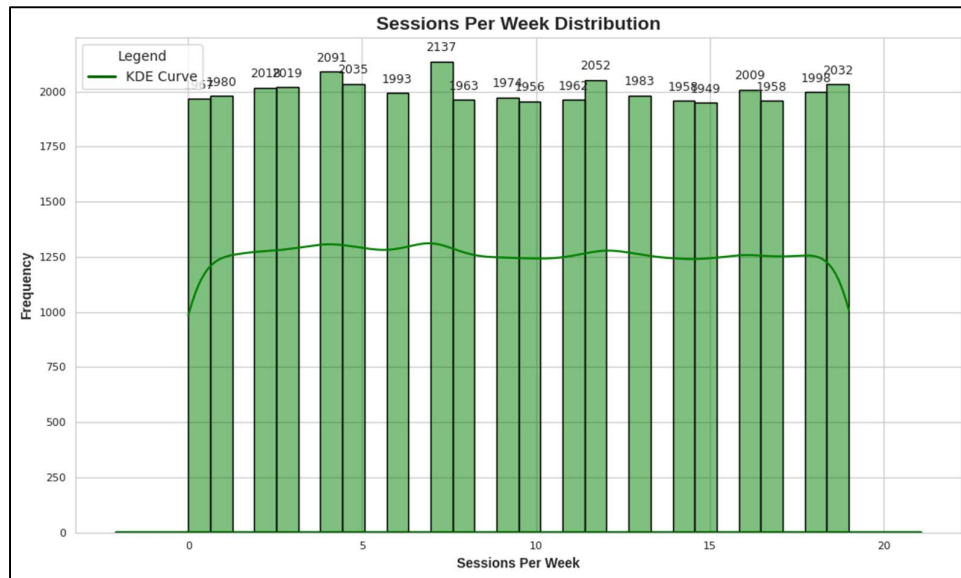


Figure-6: Distribution- Gaming Sessions per week (Histogram)

Distribution- Play Time Hours of Players

Gaming session durations were mostly uniform, with 5-20 hours per session observed in around 1300-1400 instances. Most players engaged in moderate-length sessions, while sessions under 5 hours or over 20 are less common. This trend highlighted a stable, predictable preference for medium-duration sessions among users.

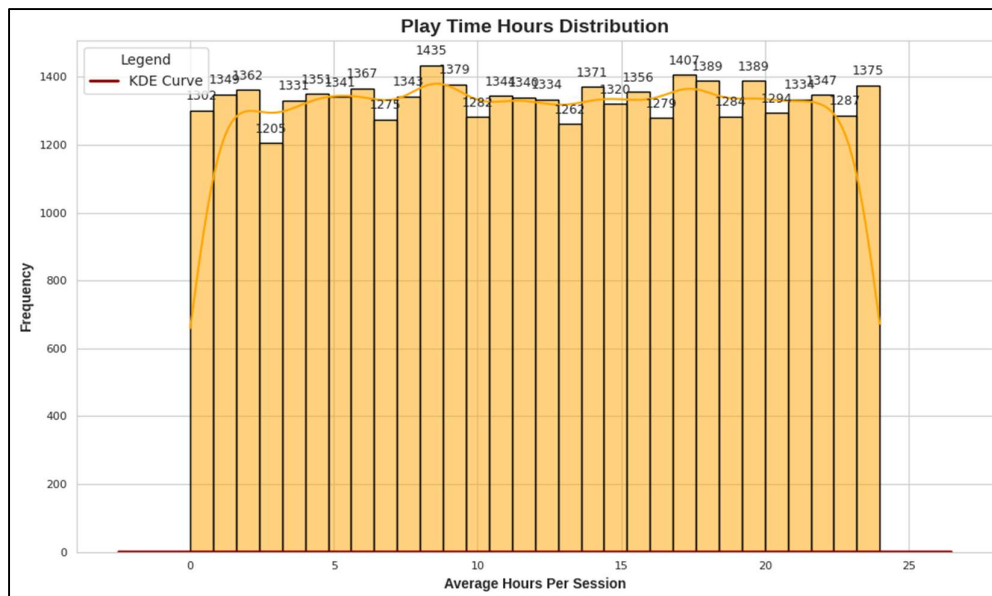


Figure-7: Distribution- Play Time Hours of Players (Histogram)

Average Session Duration by Engagement Level

From the below box plot, we observed that highly engaged players generally have longer sessions, with a median around 140 minutes. Medium engagement shows a shorter median around 90 minutes, while low engagement has the shortest median, around 65 minutes.

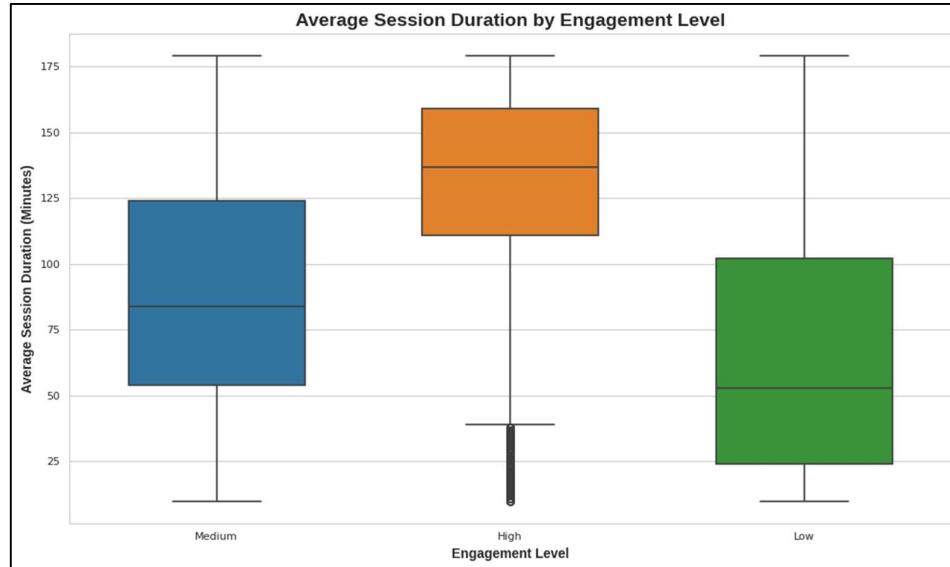


Figure-8: Average Session Duration by Engagement Level-Box Plot

In-Game Purchases vs Engagement Level- Counter Plot

We saw that a higher proportion of medium and high engagement players make in-game purchases compared to low engagement players. Additionally, the number of purchases increases with higher engagement levels. This suggested that engagement level is a key factor influencing in-game purchasing behavior.

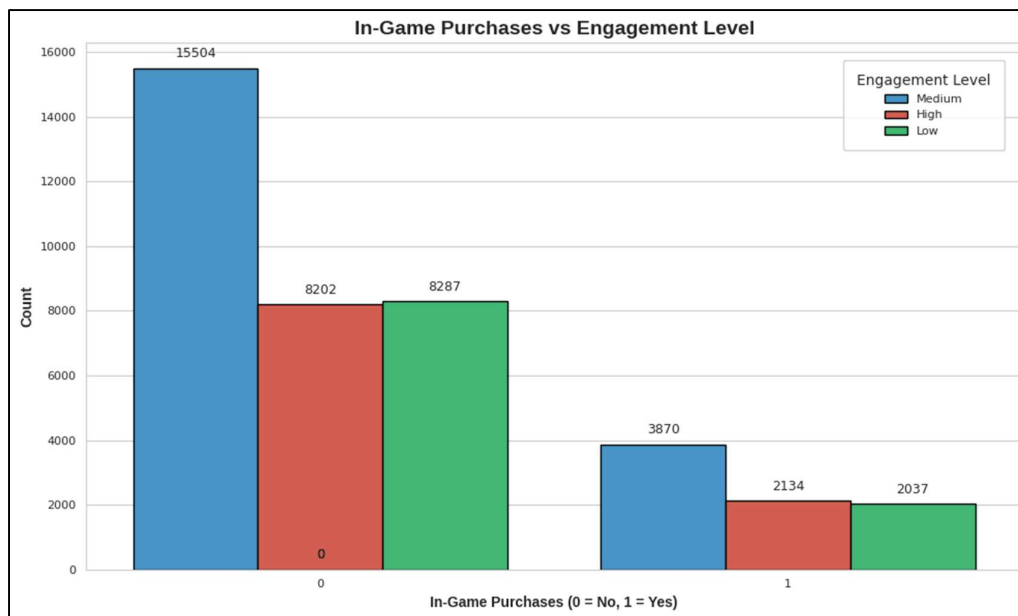


Figure-9: In-Game Purchases vs Engagement Level- Counter Plot

Correlation Matrix (Heat Map)

The heatmap below showed us that session frequency (0.49) and average session duration (0.45) are the strongest predictors of engagement. Achievements unlocked moderately correlate with engagement, while in-game purchases, gender, and age have little impact. Frequent, longer sessions stand out as key factors in predicting engagement.

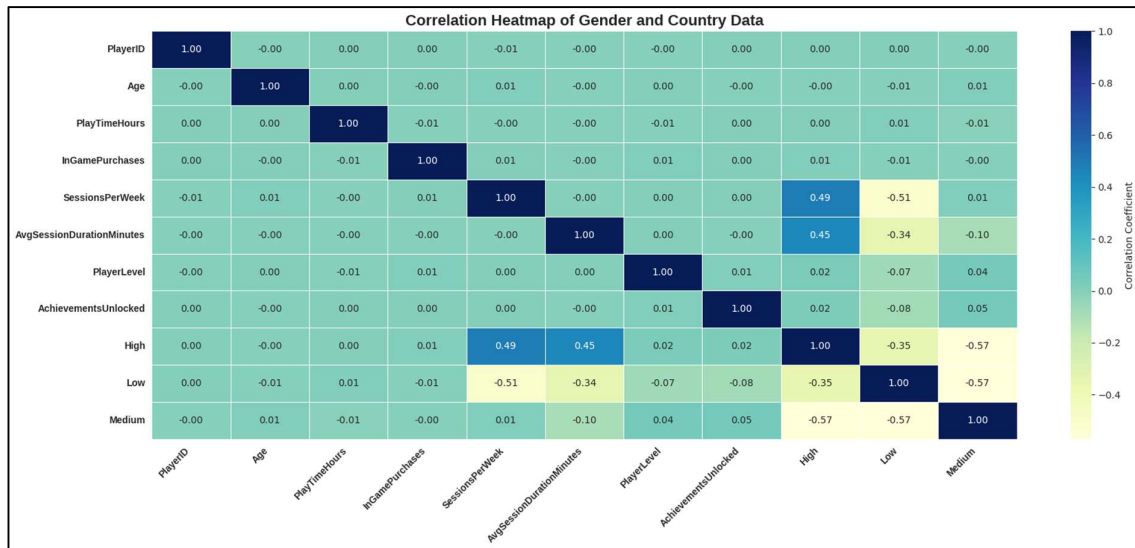


Figure-10: Correlation Map

Data Preparation

Data Cleaning

The first step in data cleaning for this dataset involved assessing missing values, duplicates, and outliers. Upon examination, we found no missing values, ensuring that our dataset was complete and ready for analysis. To visually confirm this, we generated a summary statistics table and a heatmap, as shown in Figures X and Y, which illustrate the absence of missing values across all attributes.

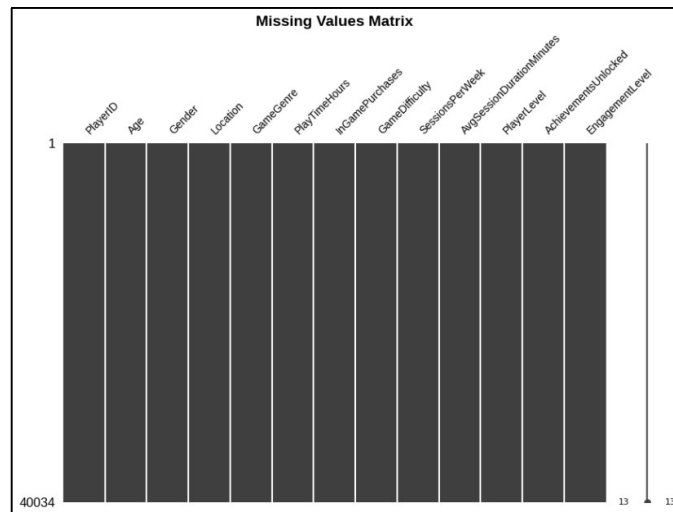


Figure-11: Missing Value Matrix



Figure-12: Missing Values Heatmap

Additionally, we verified that there were no duplicate values in the dataset, further affirming the integrity of our data. This was confirmed using a function that checks for duplicates, which revealed zero instances. Figure A displays the results of this check, emphasizing that our dataset consists of unique entries.

```
# Removing duplicate rows from the dataset

def drop_duplicates(df):
    # Check for duplicate rows
    duplicate_rows = df[df.duplicated()]

    if duplicate_rows.empty:
        print("No duplicate rows found!")
    else:
        print("Duplicate Rows:")
        print(duplicate_rows)

    # Drop duplicate rows
    df.drop_duplicates(inplace=True)
    print("Duplicate rows have been dropped.")

# Call the function with your DataFrame
drop_duplicates(df)
```

➡ No duplicate rows found!

Figure-13: Checking for Duplicates

While we explored the potential presence of outliers using boxplots for various attributes, no significant outliers were detected in the dataset. After careful review, we confirmed that the data points were within reasonable ranges, and no outliers needed to be addressed or removed. This allowed us to proceed with the analysis without further modifications, ensuring that all player behavior data could contribute meaningfully to predicting in-game recommendations and retention strategies.

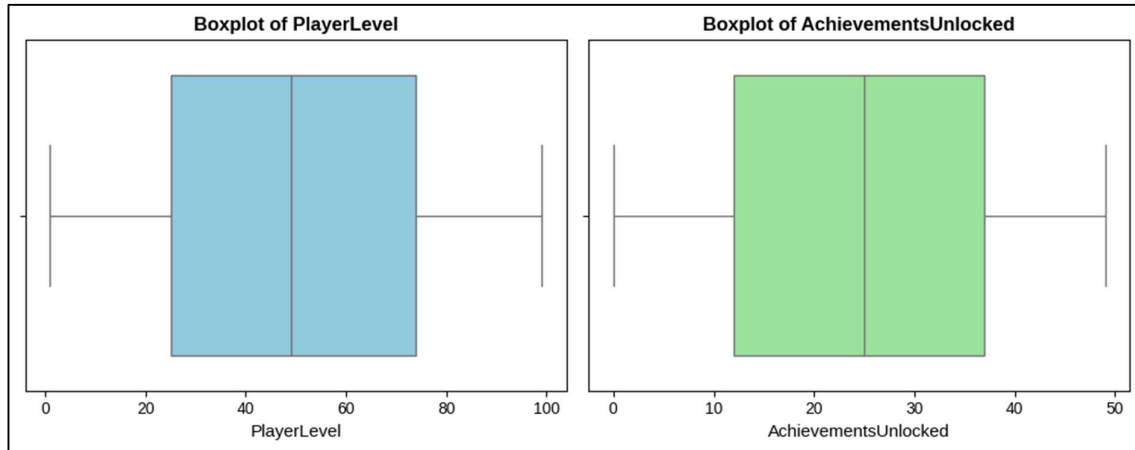


Figure-14: Boxplots (PlayerLevel & AchievementUnlocked)

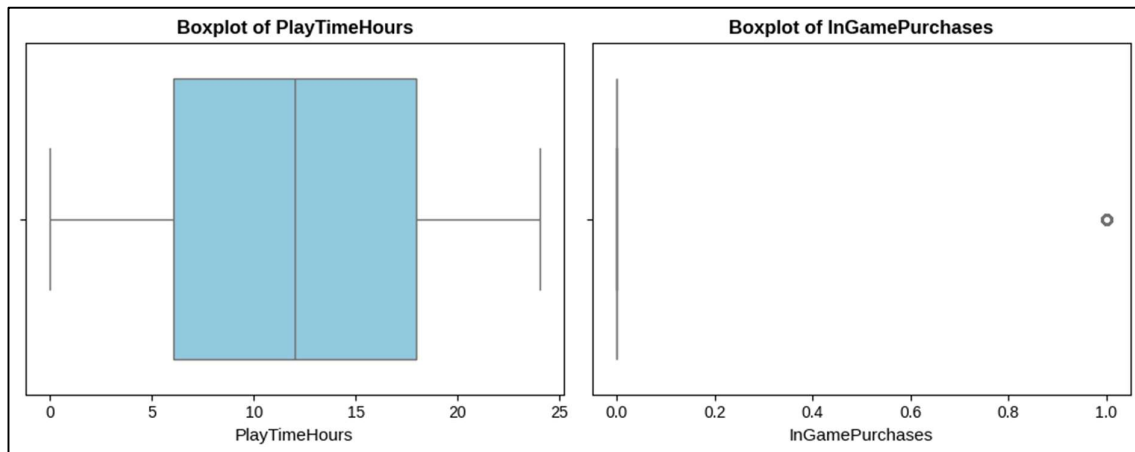


Figure-15: Boxplots (PlayTimeHours & InGamePurchases)

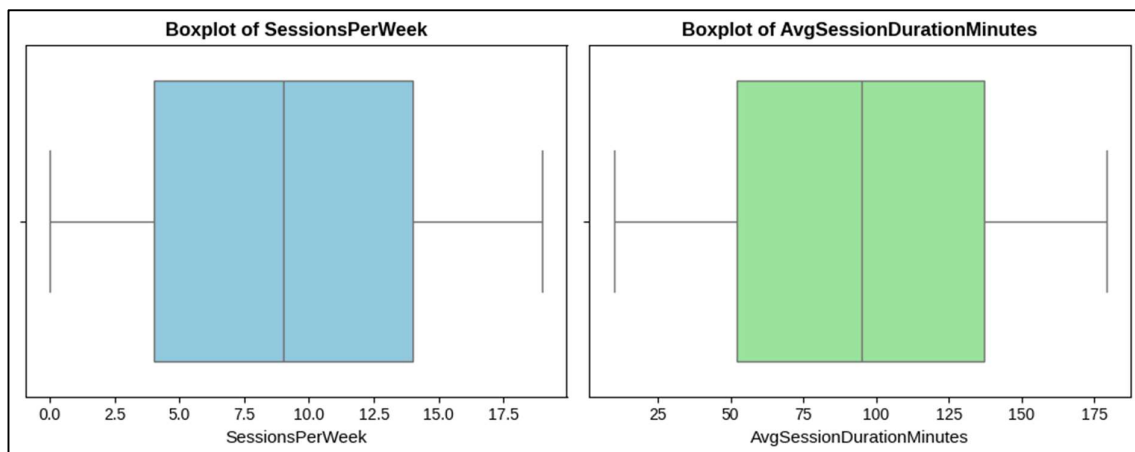


Figure-16: Boxplots (SessionsPerWeek & AvgSessionDurationMinutes)

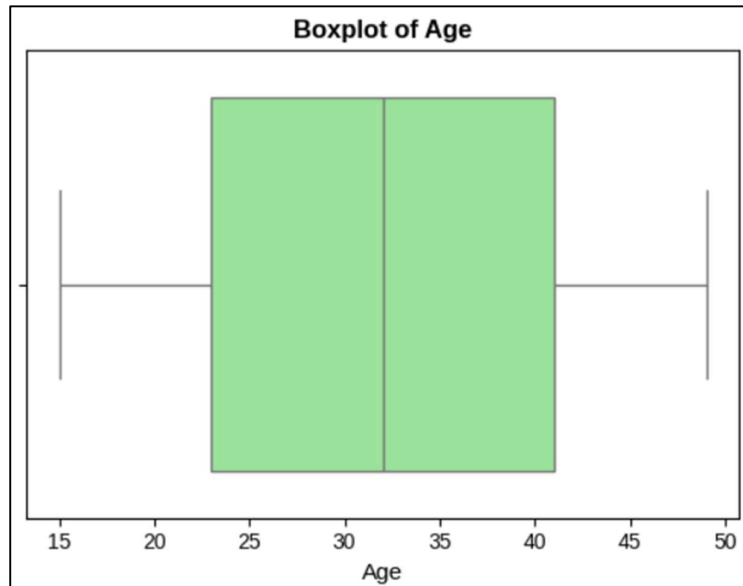


Figure-17: Boxplot (Age)

The next step involved removing unnecessary features from the dataset. After creating two separate lists for categorical and numerical data, we identified that the columns **PlayerID** and **AgeGroup** did not contribute any discriminatory power for model development. Therefore, we deemed it reasonable to drop these columns. As a result, the updated dataset now contains 12 variables, enhancing its relevance for our analysis and modeling efforts.

```
# Removing irrelevant columns from the dataset

def drop_columns(df_cleaned, columns_to_drop):
    df_cleaned.drop(columns_to_drop, axis=1, inplace=True)
    return df_cleaned

columns_to_drop = ['PlayerID', 'AgeGroup']

df_cleaned = drop_columns(df_cleaned, columns_to_drop)

df_cleaned.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 40034 entries, 0 to 40033
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                    40034 non-null  int64
1   Gender                                40034 non-null  object
2   Location                              40034 non-null  object
3   GameGenre                             40034 non-null  object
4   PlayTimeHours                         40034 non-null  float64
5   InGamePurchases                       40034 non-null  int64
6   GameDifficulty                         40034 non-null  object
7   SessionsPerWeek                       40034 non-null  int64
8   AvgSessionDurationMinutes             40034 non-null  int64
9   PlayerLevel                           40034 non-null  int64
10  AchievementsUnlocked                  40034 non-null  int64
11  EngagementLevel                       40034 non-null  object
dtypes: float64(1), int64(6), object(5)
```

Figure-18: Removing Unnecessary Features

Feature Engineering

Feature engineering is a crucial step in the data preparation process, involving the creation of new features or the modification of existing ones to enhance the predictive power of machine learning models. By transforming raw data into a format that better captures the underlying patterns, feature engineering allows algorithms to learn more effectively, ultimately leading to improved model performance and insights.

Need for Feature Engineering in Our Dataset

Feature engineering is vital for our dataset as it improves model performance by creating relevant features that capture player behavior patterns. Introducing features like Age Group, IsStressed, and GameDifficultyQuantified enabled better understanding of player demographics and stress levels, leading to more accurate predictions of engagement and in-game purchases.

To enhance our dataset and gain deeper insights into player behavior, we created the following features:

- **Age Group:** This categorizes users into age ranges (e.g., teens, adults) based on their age, helping analyze how age influences engagement patterns.
- **IsStressed:** A binary feature indicating if a player showed signs of stress during gameplay, derived from behavior metrics and feedback, which aids in analyzing the impact of stress on engagement.
- **GameDifficultyQuantified:** This quantifies game difficulty based on user ratings and performance data, allowing us to evaluate the relationship between difficulty and player retention. (1-Easy, 2-Medium, 3-Hard)

```
# Create a new column 'IsStressed' with boolean values based on 'GameDifficulty' being "Hard"
df_cleaned["IsStressed"] = np.where(df_cleaned["GameDifficulty"] == "Hard", True, False)

# Map the 'GameDifficulty' categories to numeric values for 'GameDifficultyQuantified'
df_cleaned["GameDifficultyQuantified"] = df_cleaned["GameDifficulty"].map({"Easy": 1, "Medium": 2, "Hard": 3})
```

Figure-19: New Features ("IsStressed" & "GameDifficultyQuantified")

Data Transformation

In our dataset, we began by transforming our target variable, EngagementLevel, into a numeric format using Label Encoding. This step was essential for enabling effective training of statistical models, as they require features to be numeric.

```
[ ] from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler
    from sklearn.feature_selection import VarianceThreshold

    le = LabelEncoder()
    df_cleaned['EngagementLevel'] = le.fit_transform(df_cleaned['EngagementLevel'])

df_cleaned.head()
```

	Age	Gender	Location	GameGenre	PlayTimeHours	InGamePurchases	SessionsPerWeek	AvgSessionDurationMinutes	PlayerLevel	AchievementsUnlocked	EngagementLevel
0	43	Male	Other	Strategy	16.271	0	6	108	79	25	2
1	29	Female	USA	Strategy	5.526	0	5	144	11	10	2
2	22	Female	USA	Sports	8.224	0	16	142	35	41	0
3	35	Male	USA	Action	5.265	1	9	85	57	47	2
4	33	Male	Europe	Action	15.532	0	2	131	95	37	2

Figure-20: Label Encoding (Target Variable: “EngagementLevel”)

Next, we ensured that all input features were also in numerical format. For the **GameGenre** column, we applied Label Encoding, resulting in the following mapping:

- Action = 0
- RPG = 1
- Simulation = 2
- Sports = 3
- Strategy = 4

```
from sklearn.preprocessing import LabelEncoder

# Initialize the LabelEncoder
label_encoder = LabelEncoder()

# Fit and transform the 'GameGenre' column
df_cleaned['GameGenre'] = label_encoder.fit_transform(df_cleaned['GameGenre'])

# Optional: If you want to check the mapping of original categories to labels
category_mapping = dict(zip(label_encoder.classes_, range(len(label_encoder.classes_))))
print("Category mapping:", category_mapping)
```

Category mapping: {'Action': 0, 'RPG': 1, 'Simulation': 2, 'Sports': 3, 'Strategy': 4}

Figure-21: Label Encoding (GameGenre)

Similarly, the Location column was transformed using Label Encoding, yielding the mapping:

- Asia = 0
- Europe = 1
- Other = 2
- USA = 3


```

from sklearn.preprocessing import LabelEncoder

# Initialize the LabelEncoder
label_encoder = LabelEncoder()

# Fit and transform the 'Location' column
df_cleaned['Location'] = label_encoder.fit_transform(df_cleaned['Location'])

# Optional: Check the mapping of original categories to labels
category_mapping = dict(zip(label_encoder.classes_, range(len(label_encoder.classes_))))
print("Category mapping:", category_mapping)

Category mapping: {'Asia': 0, 'Europe': 1, 'Other': 2, 'USA': 3}

```

Figure-22: Label Encoding (PlayerLocation)

These transformations ensured that our dataset was refined and ready for analysis, consisting of relevant features that accurately represented player behavior and engagement.

Feature Selection

Feature selection is a critical step in machine learning that involves identifying and retaining the most relevant features while eliminating redundant or irrelevant ones. This process is important for several reasons:

- **Improved Model Performance:** Selecting important features reduces model complexity, leading to faster training times and better performance.
- **Reduction of Overfitting:** Fewer features decrease the risk of overfitting by minimizing noise in the data, enhancing predictions on unseen data.
- **Enhanced Interpretability:** A smaller set of relevant features makes the model easier to understand, clarifying the factors driving predictions.

To enhance predictive power and focus on key variables, we applied a series of feature selection techniques aimed at identifying and retaining features most related to player engagement while eliminating redundant ones. The steps taken included:

Correlation Matrix: We utilized a correlation matrix to identify features strongly related to “EngagementLevel”. This analysis allowed us to retain only those features that exhibited meaningful correlations, guiding our subsequent steps.

```
[51] numerical_col = [i for i in df.columns if df[i].dtype != 'object']

    corr_relation = df[numerical_col].corr()

    k = 13 # We have Set the desired number as 13 which includes all features to keep

    if 'EngagementLevel' in corr_relation.columns:
        features_to_keep = corr_relation['EngagementLevel'].sort_values(ascending=False).index[:k]
    else:
        print("Error: 'EngagementLevel' column not found in DataFrame.")

[53] features_to_keep

Index(['EngagementLevel', 'AchievementsUnlocked', 'PlayerLevel', 'Age',
      'Gender', 'Location', 'GameGenre', 'GameDifficultyQuantified',
      'InGamePurchases', 'IsStressed', 'PlayTimeHours', 'SessionsPerWeek',
      'AvgSessionDurationMinutes'],
      dtype='object')
```

Figure-23: Results from Correlation Matrix

Results from Correlation Matrix: We identified 13 valuable features for analysis: **Age, Gender, Location, GameGenre, PlayTimeHours, InGamePurchases, SessionsPerWeek, AvgSessionDurationMinutes, PlayerLevel, AchievementsUnlocked, EngagementLevel, IsStressed, and GameDifficultyQuantified.**

Variance Threshold: This technique removes features with low variance, as they offer little variability and contribute minimal information for predictions. By eliminating these features, we enhance the dataset's overall quality and relevance.

Chi-Squared Test: This statistical test evaluates the independence of categorical features. By calculating p-values, we identify features with significant relationships to the target variable, selecting only those with p-values below 0.05. This ensures that only statistically significant features remain in the dataset.

	Feature	Chi-Squared Score	P-Value
6	SessionsPerWeek	51588.583	0.000
7	AvgSessionDurationMinutes	240139.968	0.000
8	PlayerLevel	3540.413	0.000
9	AchievementsUnlocked	1976.664	0.000
4	PlayTimeHours	14.505	0.001
0	Age	9.505	0.009
5	InGamePurchases	2.383	0.304
3	GameGenre	2.039	0.361
10	IsStressed	1.806	0.405
11	GameDifficultyQuantified	0.494	0.781
1	Gender	0.474	0.789
2	Location	0.419	0.811

Figure-24: Results for Final Features from Chi-Square Test

Final Feature Set: After applying the variance threshold and the chi-squared test, we compiled a new DataFrame containing the most relevant features. This final set focuses on attributes that strongly relate to player engagement, optimizing our analysis and predictive modeling efforts.

```
[60] alpha = 0.05

# Identify the target variable
target_variable = 'EngagementLevel'

# Filter features to exclude the target variable
features_to_keep = [col for col in df.columns if col != target_variable]

# Assuming you have already computed p_values for the features in features_to_keep
# Select features with p-value less than alpha
significant_features_indices = np.where(p_values < alpha)[0]

# Map the indices back to feature names to get the actual feature names
significant_feature_names = [features_to_keep[i] for i in significant_features_indices]

print("Significant features:", significant_feature_names)
```

Significant features: ['Age', 'PlayTimeHours', 'SessionsPerWeek', 'AvgSessionDurationMinutes', 'PlayerLevel', 'AchievementsUnlocked']

Figure-25: Results for Final Features from Chi-Square Test

Model Selection

Since our dataset focuses on predicting the engagement level of players, this task was classified as a multiclass classification problem (We have three levels of Engagement Levels- Low, Medium, High). The objective is to determine whether a player falls into a specific engagement category and to identify the key factors influencing this classification.

To tackle this challenge, we employed various machine learning algorithms, specifically Logistic Regression, K-Nearest Neighbour and Random Forest Classifier. These models were chosen for their interpretability and robust performance in analyzing player behavior.

Logistic Regression

Logistic regression is a widely used classification model that assumes a linear relationship between input features and the log-odds of the target variable, 'EngagementLevel.' This parametric algorithm uses maximum likelihood estimation (MLE) to quantify each feature's impact on player engagement, offering insights into both direction and magnitude. As a benchmark for more complex models, it is valued for its interpretability. However, it requires careful feature engineering and is sensitive to missing values, necessitating thorough dataset preprocessing for accurate predictions.

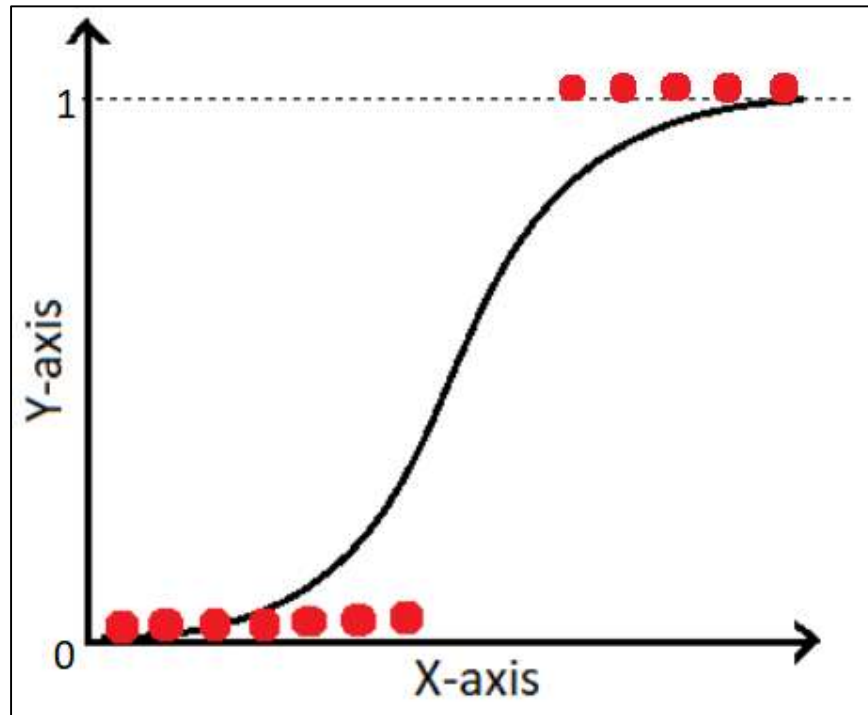


Figure-26: Logistic Regression

K-Nearest Neighbour

KNN is a non-parametric classification algorithm that predicts engagement levels by identifying the 'k' closest data points. It assumes that similar players have similar engagement levels, allowing for flexibility with different datasets. However, KNN can be computationally intensive with large datasets and is sensitive to the choice of 'k' and irrelevant features, emphasizing the need for effective feature selection and scaling.

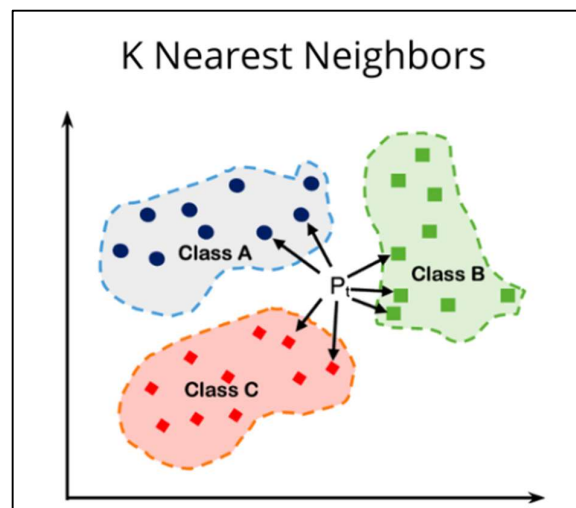


Figure-27: K-Nearest Neighbour

Random Forest Classifier

The Random Forest Classifier is an ensemble learning method that combines multiple decision trees to enhance predictive accuracy and control overfitting. It operates by training several trees on random subsets of the data and features, then aggregating their predictions. This approach improves robustness and handles non-linear relationships effectively. While it can manage large datasets and complex interactions between features, it may sacrifice interpretability compared to simpler models. Thus, careful consideration of feature importance is essential for understanding the factors influencing player engagement.

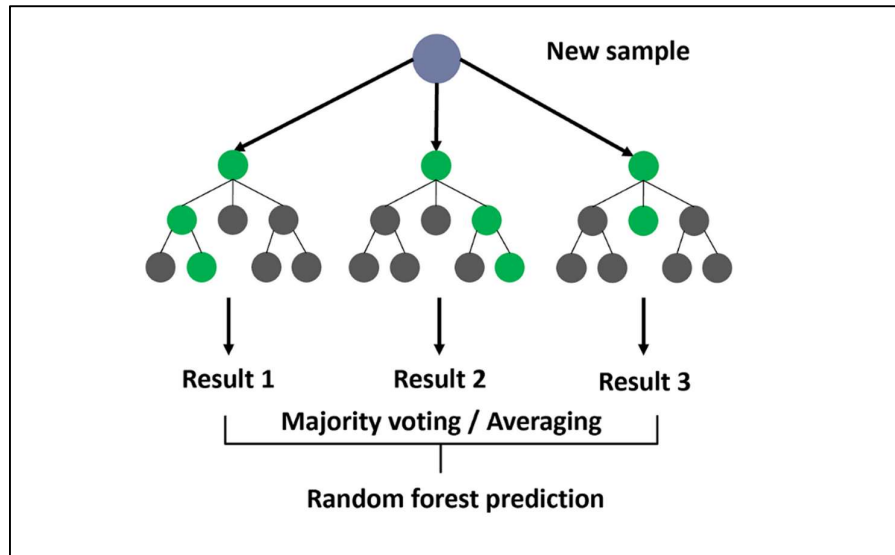


Figure-28: Random Forest Classifier

In selecting the appropriate machine learning algorithms for predicting player engagement levels, we considered various factors, including interpretability, robustness, and the ability to capture complex relationships within the dataset. The following table summarizes the chosen models and the rationale behind each selection.

Model	Justification for Selection
Logistic Regression	- Serves as a benchmark for performance evaluation. - Offers clear insights into the relationship between player attributes and engagement.
K-Nearest Neighbours (KNN)	- Simple to implement and interpret. - Effective for datasets with complex relationships and can capture local patterns in player engagement.
Random Forest Classifier	- Handles large datasets with high dimensionality well. - Captures complex interactions between features and improves predictive performance.

Table-5: Table for Justification for Models

Experiments

Experiment Plan

Our experiment aimed to predict player behavior using historical data to enhance in-game recommendations and retention strategies, ultimately improving user experience by tailoring content and offers to individual player preferences.

This section outlines our systematic approach, covering the preparation of historical player data, the development and training of predictive models, and the evaluation metrics used to assess the accuracy and effectiveness of predicting player behavior, engagement, and in-game purchases.

The following table details the structured approach taken for this experiment.

Method Involved for Experiment	Approach Taken
Dataset Preparation	Conducted thorough data preprocessing by checking for duplicates (none found), handling null values, removing unnecessary columns, and engineering features to include "IsStressed," "GameDifficultyQuantified," and an age group column derived from the age column.
Features Selection	Identified top features correlated with "EngagementLevel" from the correlation matrix, removed low-variance features using Variance Threshold, applied chi-squared test to select features with p-values < 0.05, and compiled a new DataFrame with relevant features for analysis.
Model Development	Logistic Regression: Used for binary classification of player behavior, K-Nearest Neighbors (KNN): Classifies player engagement by proximity, and Random Forest Classifier (RFC): Improves accuracy through ensemble learning of decision trees.
Model Training	Trained models on historical player behavior data, utilizing parameter tuning, regularization, and cross-validation to prevent overfitting and enhance performance; optimized hyperparameters for Logistic Regression, KNN, and Random Forest classifiers to improve accuracy.
Model Validation & Evaluation	Assessed model performance using a validation dataset, employed metrics like accuracy, precision, recall, and F1-score, analyzed confusion matrices to interpret results, and conducted ROC-AUC analysis for model robustness. Compared results across algorithms to find the best-performing model.

Table-6: Methodology Followed for this experiment (Online Gaming Behaviour Prediction)

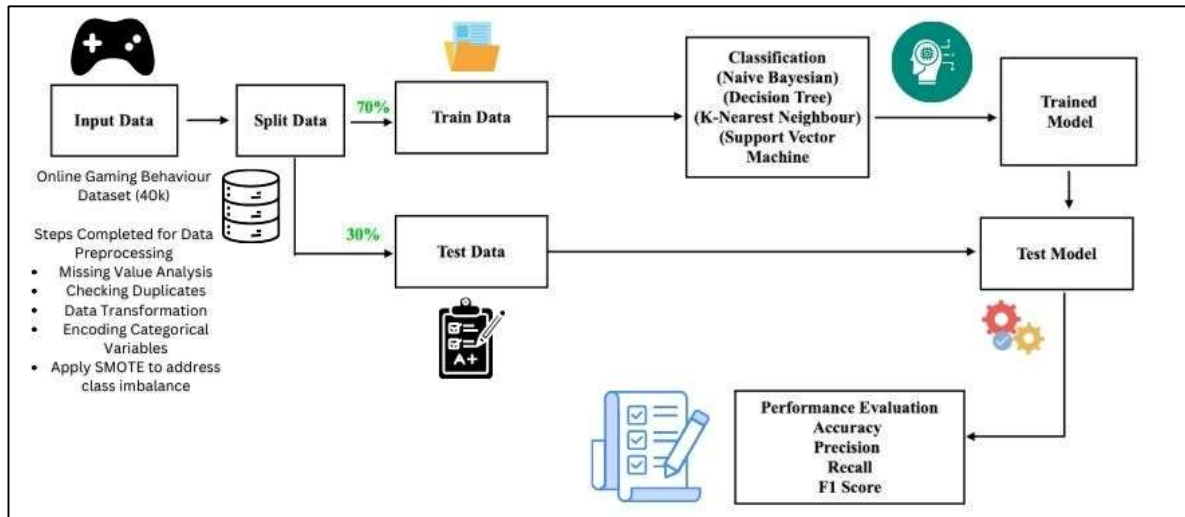


Figure-29: Process Flow for Experiments

Splitting Dataset

After selecting the final features using chi-square tests, the dataset was split into training, validation, and test sets, following an 80-20 split. We used 80% for training and validation and kept 20% for testing. Stratified sampling ensured class balance in each split.

```

from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE

# Perform train-validation-test split
X_train_val, X_test, y_train_val, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(X_train_val, y_train_val, test_size=0.25, random_state=42, stratify=y_train_val)

# Perform oversampling using SMOTE on the training data only
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)
  
```

Figure-30: Splitting of Dataset

Applying SMOTE to address class imbalance

To enhance model performance, SMOTE (Synthetic Minority Over-sampling Technique) was applied to the training data. This technique generated synthetic samples for underrepresented classes, addressing class imbalance and creating a more robust dataset for our predictive models.

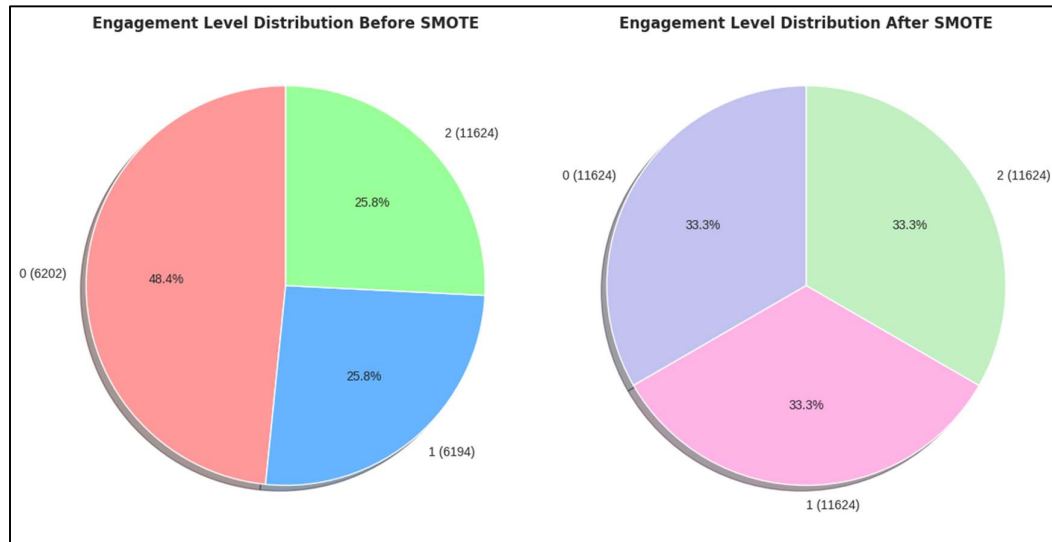


Figure-31: Applying SMOTE to address class imbalance

Model Development & Training

We trained three models—Logistic Regression, KNN, and Random Forest—using significant features identified via chi-square tests, focusing on predicting three engagement levels ('low,' 'medium,' 'high').

For this multiclass classification problem, predicting three engagement levels, we used these performance indicators:

- **Accuracy:** Measured overall correctness, though less reflective due to class imbalance.
- **Precision:** Assessed prediction accuracy, crucial for minimizing false positives, especially for 'high' engagement.
- **Recall:** Captured relevant instances of each engagement level ('low,' 'medium,' 'high').
- **F1 Score:** Balanced precision and recall, helpful for imbalanced classes.
- **ROC AUC:** Using OvR, ROC curves highlighted how well the model separated the three classes, with higher AUC indicating better class distinction.

Hyperparameter Tuning

To optimize the performance of our machine learning models, we employed hyperparameter tuning techniques. Specifically, we utilized **GridSearchCV** for the Logistic Regression and K-Nearest Neighbors (KNN) algorithms, which allows for an exhaustive search over specified parameter values to find the best combination for improving model accuracy. On the other hand, we opted for **RandomizedSearchCV** for the Random Forest Classifier (RFC), as it enables a more efficient search by randomly sampling parameter combinations, which is particularly beneficial for large parameter spaces. This approach strikes a balance between computational efficiency and model performance, ensuring that we achieve the best possible results for each algorithm.

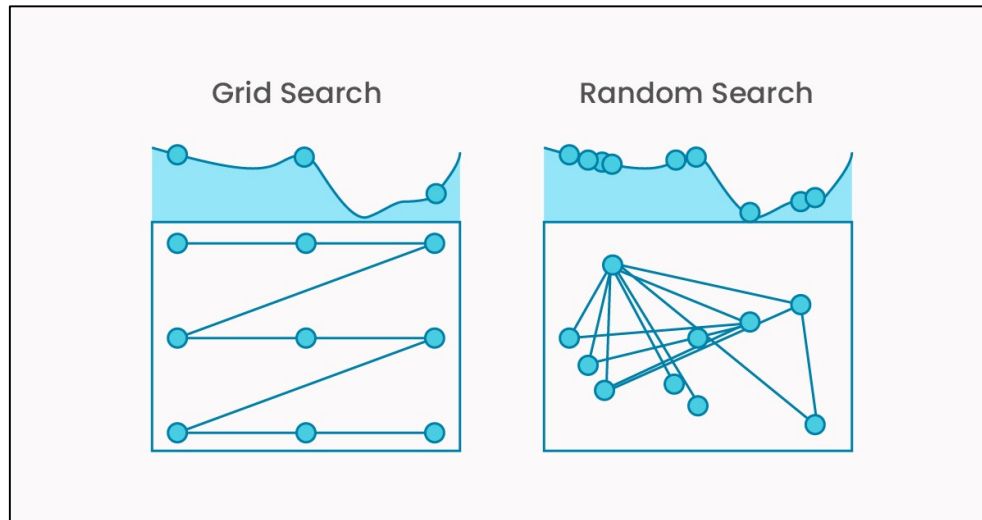


Figure-32: Grid Search & Random Search

Stratified K-fold Cross Validation

We used Stratified K-Fold cross-validation to ensure balanced class distribution. After tuning hyperparameters with GridSearchCV and RandomizedSearchCV, the model was trained and validated over 10 folds. For each fold, metrics like accuracy, F1 score, precision, recall, and ROC AUC were calculated. We also tracked the best model's performance across all folds by updating metrics when a fold produced a higher score.

```
=====
Starting with fold: 1/10
Logistic Regression - Accuracy: 0.7922, F1: 0.7919, Precision: 0.8014, Recall: 0.7922
KNN - Accuracy: 0.7892, F1: 0.7894, Precision: 0.7940, Recall: 0.7892
Random Forest - Accuracy: 0.9183, F1: 0.9182, Precision: 0.9182, Recall: 0.9183
=====
Starting with fold: 2/10
Logistic Regression - Accuracy: 0.7942, F1: 0.7937, Precision: 0.8028, Recall: 0.7942
KNN - Accuracy: 0.7922, F1: 0.7922, Precision: 0.7974, Recall: 0.7922
Random Forest - Accuracy: 0.9226, F1: 0.9224, Precision: 0.9226, Recall: 0.9226
=====
Starting with fold: 3/10
Logistic Regression - Accuracy: 0.7890, F1: 0.7881, Precision: 0.7977, Recall: 0.7890
KNN - Accuracy: 0.7907, F1: 0.7907, Precision: 0.7975, Recall: 0.7907
Random Forest - Accuracy: 0.9198, F1: 0.9196, Precision: 0.9197, Recall: 0.9198
=====
Starting with fold: 4/10
Logistic Regression - Accuracy: 0.7880, F1: 0.7880, Precision: 0.7986, Recall: 0.7880
KNN - Accuracy: 0.7827, F1: 0.7830, Precision: 0.7870, Recall: 0.7827
Random Forest - Accuracy: 0.9153, F1: 0.9152, Precision: 0.9152, Recall: 0.9153
=====
Starting with fold: 5/10
Logistic Regression - Accuracy: 0.7857, F1: 0.7853, Precision: 0.7972, Recall: 0.7857
KNN - Accuracy: 0.7832, F1: 0.7832, Precision: 0.7893, Recall: 0.7832
Random Forest - Accuracy: 0.9168, F1: 0.9167, Precision: 0.9167, Recall: 0.9168
```

Figure-33: Stratified K-Fold (Example- 5 Folds for Test Set)

Baseline Model

The baseline model, predicting the most common class, achieved 48.39% accuracy across all sets, relying solely on the dominant class without using input features. This benchmark establishes a minimum standard that more advanced models should exceed, highlighting the importance of features in enhancing predictive performance.

Baseline Model	Accuracy %
Training Set	48.39%
Validation Set	48.39%
Test Set	48.39%

Table-7: Baseline Model Performance

Logistic Regression

The Logistic Regression model was applied to the dataset after selecting the final features from the feature selection process. The model was trained using these features, and the results on both the validation and test sets were obtained without any hyperparameter tuning.

Model (Logistic Regression)	Accuracy %	Precision	Recall	F1-Score	ROC Curve
Validation	0.7816	0.7910	0.7816	0.7812	N/A
Test	0.7869	0.7972	0.7869	0.7866	N/A

Table-8: Performance Results (Logistic Regression- Before Hyperparameter Tuning)

To optimize model performance, we used GridSearchCV for hyperparameter tuning in the Logistic Regression model, resulting in the best hyperparameters given below. After using the parameters received during hypermeter tuning, we got the following performance results given below:

```
Best Logistic Regression Model: LogisticRegression(C=0.1, class_weight='balanced', max_iter=10000, penalty='l1', solver='saga', warm_start=True)
```

Figure-34: Best Parameters (GridSearchCV- Logistic Regression)

After tuning the Logistic Regression model with GridSearchCV, we applied 10-fold Stratified K-Fold cross-validation for balanced class distribution and evaluated performance on the validation and test sets.

Model (Logistic Regression)	Accuracy %	Precision	Recall	F1-Score	ROC Curve
Validation	0.7832	0.7926	0.7832	0.7832	0.9083
Test	0.7942	0.8028	0.7942	0.7937	0.9133

Table-9: Performance Results (Logistic Regression- After Hyperparameter Tuning)

From the results, we can see that after hyperparameter tuning, the Logistic Regression model's performance improved in all metrics, with accuracy increasing from 78.69% to 79.42%, and F1-score rising from 0.7866 to 0.7937 on the test set.

Also ,we generated the confusion matrix and also ROC Curve for our testing set that we got from our best model given as follows:

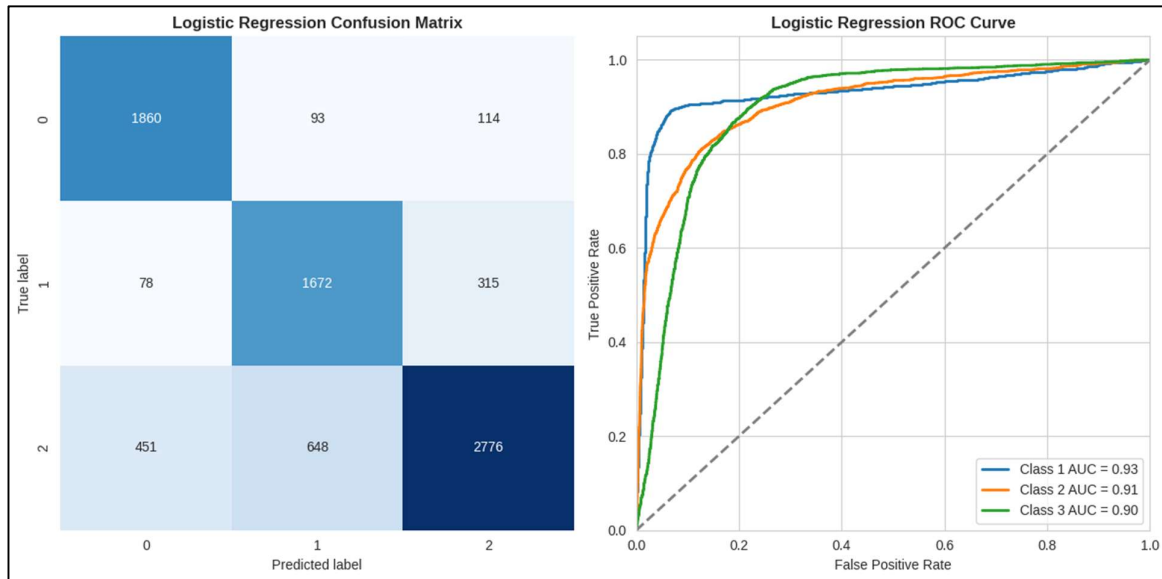


Figure-35: Confusion Matrix & ROC Curve (Logistic Regression)

The logistic regression model effectively classifies online gaming behavior into low, medium, and high engagement levels. The model demonstrates strong performance for identifying high-engagement players (Class 2), with an AUC of 0.91. However, it exhibits moderate accuracy in distinguishing between low (Class 0) and medium (Class 1) engagement levels.

K-Nearest Neighbour (KNN)

The K-Nearest Neighbors (KNN) model was applied to the dataset after final feature selection. The model was trained using these features, and the initial results on both the validation and test sets were obtained without any hyperparameter tuning.

Model (K-Nearest Neighbour)-Before Hyperparameter Tuning	Accuracy %	Precision	Recall	F1-Score	ROC Curve
Validation	0.7252	0.7334	0.7252	0.7247	0.8645
Test	0.7331	0.7409	0.7331	0.7328	0.8693

Table-10: Performance Results (KNN- Before Hyperparameter Tuning)

To enhance model performance, we utilized GridSearchCV for hyperparameter tuning in the KNN model, resulting in the best hyperparameters outlined below

```
Best KNN Model: KNeighborsClassifier(n_neighbors=7, p=1, weights='distance')
```

Figure-36: Best Parameters (GridSearchCV-KNN)

Following the tuning of the KNN model with GridSearchCV, we implemented 10-fold Stratified K-Fold cross-validation to ensure balanced class distribution and evaluated performance on the validation and test sets.

Model (K-Nearest Neighbour)-After Hyperparameter Tuning	Accuracy %	Precision	Recall	F1-Score	ROC Curve
Validation	0.7683	0.7743	0.7683	0.7682	0.8921
Test	0.7922	0.7974	0.7922	0.7922	0.9066

Table-11: Performance Results (KNN- After Hyperparameter Tuning)

The K-Nearest Neighbors (KNN) model applied to the online gaming behavior dataset showed significant improvement after hyperparameter tuning. Before tuning, it achieved 72.52% accuracy on the validation set and 73.31% on the test set, with F1-scores of 0.7247 and 0.7328. After tuning, accuracy on the test set rose to 79.22%, with an F1-score of 0.7922. Precision and recall also improved, indicating better identification of engagement levels. Additionally, ROC curve scores increased from 0.8693 to 0.9066, reflecting enhanced ability to predict player engagement.

We also generated the confusion matrix and ROC curve for the testing set using the best-performing KNN model, as detailed below:

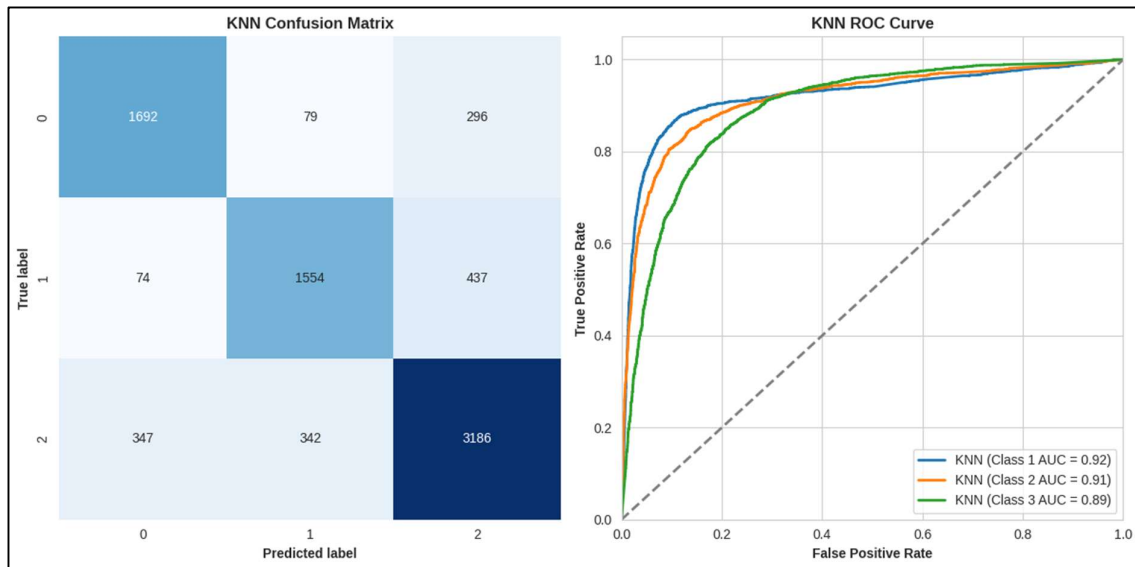


Figure-37: Confusion Matrix & ROC Curve (KNN)

The K-Nearest Neighbors (KNN) model's confusion matrix shows balanced performance across all classes. Class 1 (low-engagement) achieved 1,692 true positives, 421 false positives, and 375 false negatives. Class 2 (medium-engagement) had 1,554 true positives, 421 false positives, and 511 false negatives. Class 3 (high-engagement) performed best with 3,186 true positives, 733 false positives, and 689 false negatives. ROC scores were 0.92, 0.91, and 0.89 for Classes 1, 2, and 3, respectively, highlighting strong predictive ability, especially for Class 1.

Random Forest Classifier

The Random Forest Classifier was applied to the dataset following the final feature selection. The model was initially trained using the selected features, and results on the validation and test sets were obtained without hyperparameter tuning.

Model (Random Forest)-Before Hyperparameter Tuning	Accuracy %	Precision	Recall	F1-Score	ROC Curve
Validation	0.9129	0.9128	0.9129	0.9128	0.9402
Test	0.9112	0.9111	0.9112	0.9111	0.9439

Table-12: Performance Results (Random Forest- Before Hyperparameter Tuning)

To improve performance, we used RandomizedSearchCV for hyperparameter tuning due to its efficiency in finding the best parameters in less time as compared with GridSearchCV.

```
Best Random Forest Model: RandomForestClassifier(bootstrap=False, max_depth=46, min_samples_split=4,
n_estimators=108, random_state=42)
```

Figure-38: Best Parameters (RandomisedSearchCV-Random Forest)

After tuning the Random Forest model with RandomizedSearchCV, we applied 10-fold Stratified K-Fold cross-validation for balanced class distribution and evaluated performance on the validation and test sets.

Model (Random Forest)- After Hyperparameter Tuning	Accuracy %	Precision	Recall	F1-Score	ROC Curve
Validation	0.9126	0.9126	0.9126	0.9124	0.9407
Test	0.9226	0.9226	0.9226	0.9224	0.9433

Table-13: Performance Results (Random Forest- After Hyperparameter Tuning)

The Random Forest model demonstrated strong performance both before and after hyperparameter tuning. Initially, it achieved an accuracy of 91.12% on the test set, with an F1-score of 0.9111 and a ROC score of 0.9439. After tuning, accuracy improved to 92.26%, with an F1-score of 0.9224, showing marginal gains in precision and recall, while the ROC score remained stable at 0.9433, reflecting consistently high discriminative ability.

We also generated the confusion matrix and ROC curve for the test set using the best-performing Random Forest model, as detailed below.

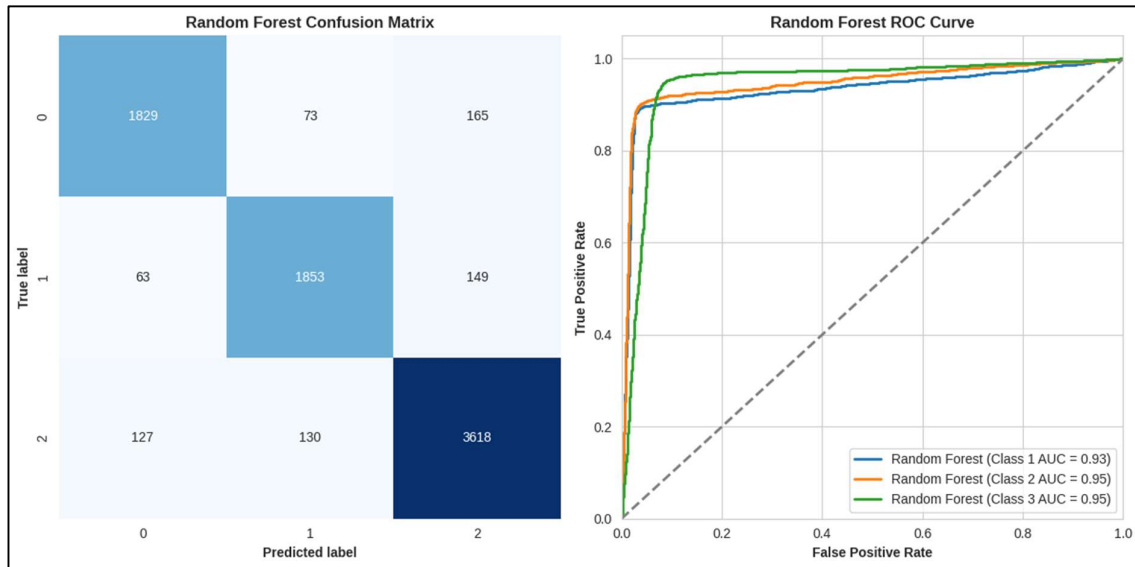


Figure-39: Confusion Matrix & ROC Curve (Random Forest)

The Random Forest Classifier shows strong performance in predicting online gaming engagement levels. The confusion matrix highlights high true positives across all classes, with some misclassifications due to false positives and negatives. The ROC curve confirms effectiveness, with AUC values of 0.93 for Class 1 and 0.95 for Classes 2 and 3. Overall, the model demonstrates its ability to distinguish between engagement levels well, though some room for improvement remains in reducing misclassifications.

Since the Random Forest model outperformed both KNN and Logistic Regression, feature importance will be analyzed to identify the key factors impacting player engagement. The top two variables influencing engagement are 'SessionsPerWeek' and 'AvgSessionDurationMinutes,' as revealed by the Mean Decrease in Gini coefficient. These features play a crucial role in predicting players' engagement levels in the online gaming dataset.

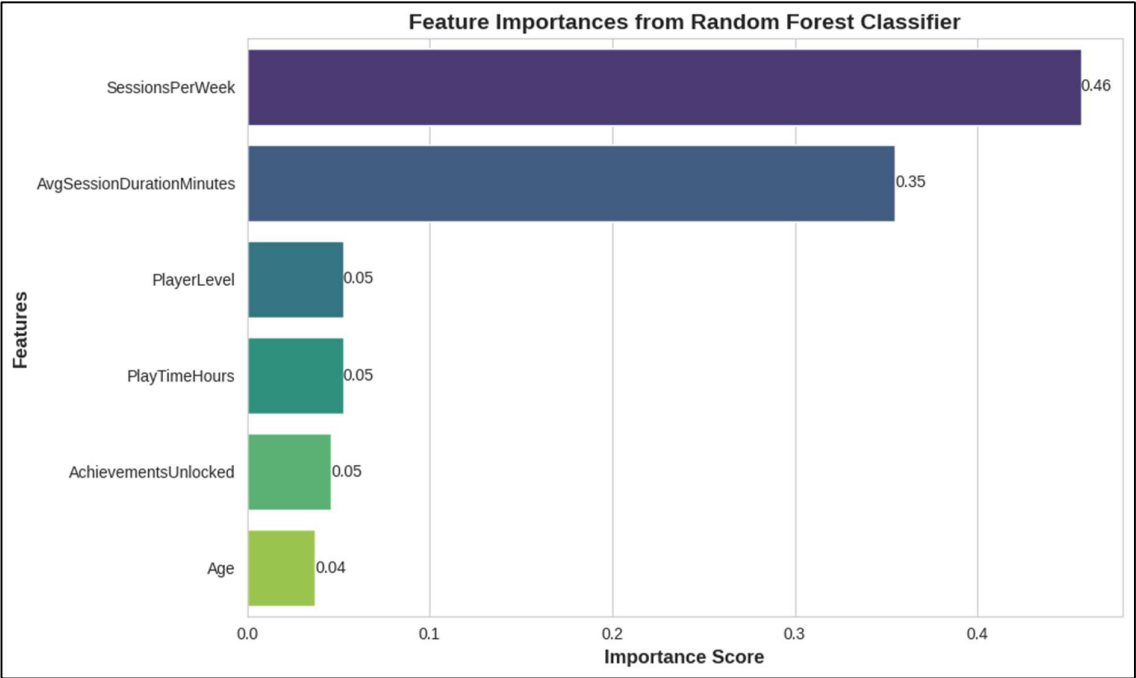


Figure-40: Feature Importance (MeanDecreaseGini)

Model Validation & Evaluation

The performance of three models—Logistic Regression, K-Nearest Neighbors (KNN), and Random Forest Classifier—was evaluated using several metrics, including accuracy, precision, recall, F1 score, and ROC-AUC.

Model Used	Baseline Model Accuracy (%)	Accuracy %	Precision	Recall	F1-Score	ROC Score
Logistic Regression	48.39%	0.7942	0.8028	0.7942	0.7937	0.9133
K-Nearest Neighbour		0.7922	0.7974	0.7922	0.7922	0.9066
Random Forest Classifier		92.26%	0.9226	0.9226	0.9224	0.9433

Table-14: Performance Results

- **Random Forest Classifier** outperformed the others, achieving an accuracy of 92.26%, significantly higher than both Logistic Regression (79.42%) and KNN (79.22%). This makes it the most reliable model for accurately classifying player engagement levels.

- In terms of **precision**, Random Forest also led with 92.26%, showing its ability to minimize false positives better than Logistic Regression (80.28%) and KNN (79.74%).
- Random Forest demonstrated the highest **recall** (92.26%), meaning it effectively identified most of the positive engagement cases. Logistic Regression and KNN had slightly lower recall scores of 79.42% each.
- For the **F1 score**, Random Forest excelled with 92.24%, indicating a strong balance between precision and recall, followed by Logistic Regression (79.37%) and KNN (79.22%).
- Finally, **ROC-AUC** showed that Random Forest had the highest score of 94.33%, illustrating superior class discrimination compared to Logistic Regression (91.33%) and KNN (90.66%).

Best Model for Classification Problem

From the below chart, we can say that **Random Forest Classifier** emerged as the best model, even without extensive hyperparameter tuning (Although we have used RandomizedSearchCV to twerk parameters to make performance better). It delivered top results in accuracy, precision, recall, F1 score, and AUC-ROC, indicating strong generalization to unseen data and effective handling of the dataset's complexity.

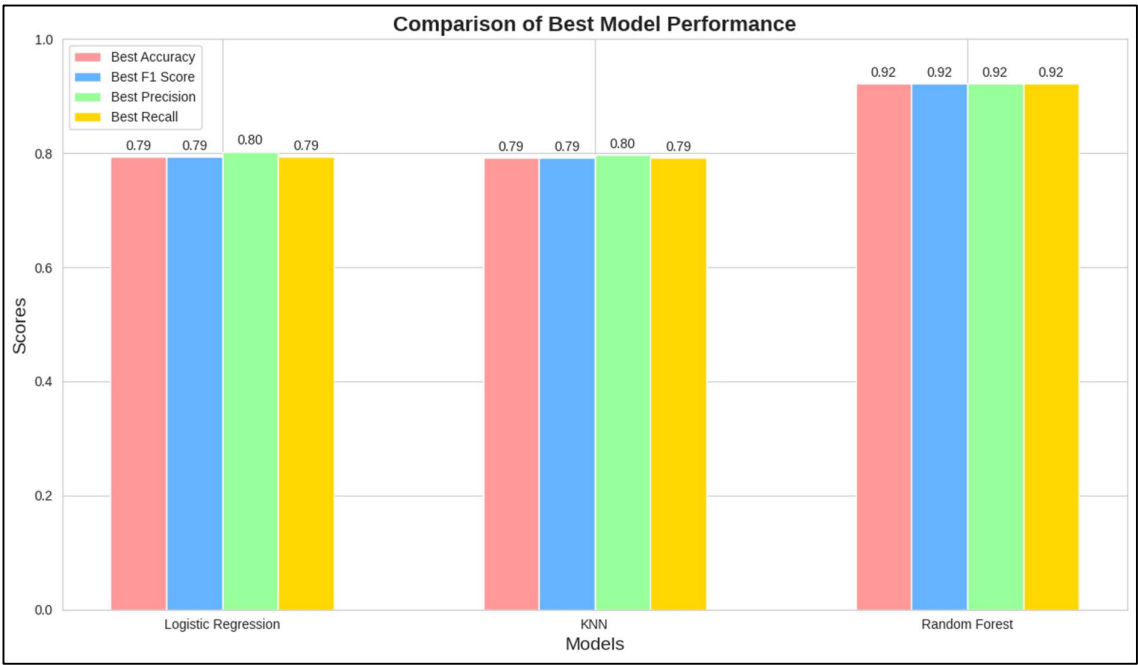


Table-41: Comparison Chart (Best Model Performance)

With an **AUC-ROC of 94.33%**, it excels in distinguishing between engagement levels, making it ideal for classification tasks requiring accurate class separation.

Given its strong performance across all metrics, **Random Forest** is well-suited for practical deployment, particularly for predicting user engagement in online gaming where reliable and balanced predictions are key.

Conclusion & Discussion

Model Performance & Key Findings

This project successfully designed and tested models to predict player engagement in online gaming, revealing the Random Forest model's superiority. With an accuracy of **92.26%** and an AUC of **94.33%** on the test set, the Random Forest model consistently outperformed Logistic Regression and KNN across metrics like precision, recall, F1-score, and ROC-AUC. This strong performance is attributed to effective feature engineering, including **feature selection and hyperparameter tuning**.

The findings highlight **SessionsPerWeek** and **AvgSessionDurationMinutes** as the most influential predictors of player engagement, providing valuable insights for game developers aiming to enhance player retention and satisfaction.

Impact on Industry Applications

The model's ability to accurately classify player engagement levels holds potential for real-world applications in online gaming platforms. By accurately indicating engagement levels, the model enables personalization of content, real-time recommendations, and targeted in-game purchases—strategies that could drive **revenue growth** by improving **user satisfaction and retention**. Game developers can leverage these insights to focus on the frequency and duration of player sessions, refining gameplay elements that promote positive engagement.

Possible Limitations

While the model achieved high predictive accuracy, certain limitations were acknowledged:

Data Scope: This analysis used data from a single gaming platform, which limited the generalization to other genres, player demographics, or engagement behaviours. Broader datasets from various gaming genres (e.g., RPG, sports, simulation) may yield insights into more diverse engagement patterns.

Ethical Concerns: The study emphasized increasing engagement, which, if applied without caution, could promote unhealthy gaming patterns. Ethical considerations are essential to ensure responsible use of predictive models to avoid promoting compulsive gaming or excessive in-game spending.

Excluded Factors: While historical behavioural data like playtime and session frequency were central, other factors—such as **psychological elements** (e.g., motivation, satisfaction) or **social aspects** (e.g., community interaction)—were not considered. Integrating these factors could further have improved model accuracy and deepen insights into player engagement.

Computational Challenges: While GridSearchCV and RandomizedSearchCV were effective for hyperparameter tuning, they were computationally expensive. Advanced optimization

techniques, such as **Bayesian Optimization**, could potentially speed up the tuning process or discover even better hyperparameters.

Future Recommendations for Experiments

Building on this project's insights, future research and implementations in online gaming could improve prediction accuracy, personalization, and ethical impact:

Richer Feature Sets: Future research can benefit from incorporating additional features beyond behavioural metrics, such as social interactions (e.g., chat frequency, group play) and psychological data (e.g., player satisfaction surveys). These could provide a deeper understanding of engagement by reflecting a broader range of player motivations and experiences, which may improve prediction performance.

Advanced Modeling Techniques: While Random Forest performed well, exploring more sophisticated models like XGBoost, LightGBM, or deep learning architectures could further raise predictive accuracy. These models are particularly effective at capturing nonlinear relationships, making them well-suited for the complex nature of player engagement data.

Cross-Genre Generalization: Expanding data sources to various genres (e.g., RPG, sports) would develop models adaptable to diverse player behaviours, enabling wider application across different gaming environments.

Real-Time Predictions: Transitioning from offline to real-time models could enable faster interventions, allowing for personalized recommendations, dynamic game difficulty adjustments, and targeted in-game purchases, boosting user engagement.

Ethical and Responsible Use: Establishing guidelines for ethical model use will ensure these predictions enhance player well-being without promoting unhealthy habits, focusing on meaningful engagement rather than compulsive play.

Regular Model Updates: Continuously updating models with new data and trends will maintain relevance and accuracy, adapting predictions to changing player behaviours and preferences.

References & Appendices

1. 🎮 predict online gaming behavior dataset. (n.d.). Kaggle: Your Machine Learning and Data Science Community. <https://www.kaggle.com/datasets/rabieelkharoua/predict-online-gaming-behavior-dataset>
2. Sachinsoni. (2023, June 11). K nearest Neighbours — Introduction to machine learning algorithms. Medium. <https://medium.com/@sachinsoni600517/k-nearest-neighbours-introduction-to-machine-learning-algorithms-9dbc9d9fb3b2>
3. Yehoshua, R. (2023, July 17). Random forests. Medium. <https://medium.com/@roiyehe/random-forests-98892261dc49>

4. SATPATHY, S. (2023, November 17). SMOTE for Imbalanced classification with Python. Analytics Vidhya. <https://www.analyticsvidhya.com/blog/2020/10/overcoming-class-imbalance-using-smote-techniques/>
5. IXie. (2024, March 25). The role of player behavior analytics in gaming. iXie Gaming. <https://www.ixiegaming.com/blog/role-of-player-behavior-analytics-in-gaming/>
6. How can machine learning help you predict user behavior? (2024, August 15). Sonamine. <https://www.sonamine.com/blog/how-can-machine-learning-help-you-predict-user-behavior>

Contribution Table

Section	Tasks	Contributors
Part A. Introduction	Business Context, Project Origins & Motivation, Project Contributions, Paper Organization	Haoran Fang, Alvaro Perico
Part B. Scoping	Project Aims, Objectives & Deliverables, Stakeholder Analysis, Success Criteria	Fabiha Rahman, Alvaro Perico
Part C. Methodology	Methodology Framework, Data Understanding, Data Preparation, Model Selection	Spenser Gautama, Qimin Yin
Part D. Experiments	Data Partitioning, Modeling & Performance Measurement, Model Comparison, Discussion & Conclusions	Paras Sharma, Qimin Yin
Part E. Appendix	Attribute Description, Contribution Table	Fabiha Rahman